UTS

PENGOLAHAN CITRA

NAMA : Gian Ananda Virja

NIM : 202431129

KELAS : F

DOSEN : Rizqia Cahyaningtyas, ST, M.Kom.

NO.PC :

ASISTEN : 1. ERIN KATHOLICA SAKRALY PUTRI

2. MORGAN IMMANUEL

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

Table of Contents

DAFTAR ISI 2
BAB I 3
PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II 4
LANDASAN TEORI 4
2.1 Pengelolahan Citra Digital dan Pembentukan Matriks Piksel 4
2.2 Perbandingan Ruang Warna RGB dan Konversi ke HSV 4
2.3 Pemisahan Objek (*Thresholding*) dan Logika *Bitwise* 4
2.4 Membaca Grafik Penyebaran Piksel (Histogram) 5
2.5 Pengubahan ke Format *Grayscale* 5
2.6. Memperbaiki Citra *Backlight* dengan Metode CLAHE 5
BAB III 7
HASIL 7
3.1 Pembahasan Soal 1 7
BAB IV 18
PENUTUP 18
DAFTAR PUSTAKA 19

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

1. *Bagaimana cara melakukan segmentasi dan ekstraksi fitur warna secara spesifik (merah, hijau, dan biru) pada citra tulisan tangan menggunakan ruang warna Hue, Saturation, Value (HSV)?*
2. *Bagaimana menganalisis efektivitas pemisahan warna dari latar belakang citra melalui visualisasi distribusi piksel menggunakan grafik histogram?*
3. *Bagaimana memperbaiki kualitas visual citra dengan masalah pencahayaan buruk (backlight) menggunakan operasi piksel adaptif tanpa menimbulkan efek hilangnya detail pada area terang (color burn)?*

1.2 Tujuan Masalah

1. *Mengimplementasikan metode thresholding pada ruang warna HSV menggunakan pustaka OpenCV di Python untuk mengisolasi warna tinta tertentu dari latar belakang kertas.*
2. *Memvisualisasikan dan mengevaluasi pergeseran intensitas piksel hasil segmentasi citra secara kuantitatif melalui pembentukan grafik histogram.*
3. *Mengaplikasikan metode Contrast Limited Adaptive Histogram Equalization (CLAHE) untuk meningkatkan kecerahan dan kontras secara lokal pada area bayangan/gelap (grayscale).*

1.3 Manfaat Masalah

1. *Memberikan pemahaman praktis mengenai keunggulan pemrosesan citra pada ruang warna HSV dibandingkan ruang warna RGB, terutama dalam menangani sensitivitas terhadap perubahan cahaya.*
2. *Meningkatkan kemampuan analitis praktikan dalam membaca karakteristik citra digital melalui grafik statistik (histogram).*
3. *Memberikan solusi komputasional yang relevan di dunia nyata untuk memperbaiki kualitas foto yang mengalami under-exposure atau backlight menggunakan algoritma pemerataan histogram yang terkontrol.*

BAB II
LANDASAN TEORI

2.1 Pengelolahan Citra Digital dan Pembentukan Matriks Piksel

Pengolahan citra digital pada dasarnyaa adalah proses memanipulasai gambar dua dimensi menggunakan algoritma komputer. Dalam dunia komputasi, komputer tidak melihat gambar seperti mata manusia, melainkan melihatnya sebagai kumpulan angka dalam sebuah matriks. Secara matematis, gambar ini dinyatakan sebagai fungsi intensitas cahaya f(x, y), di mana x dan y adalah titik kordinat baris dan kolom. Titik kordinat inilah yang biasa kita sebut sebagai piksel (picture element).Supaya gambar dari dunia nyata bisa diproses oleh komputer, gambar tersebut harus melewati proses digitalisasi yang terdiri dari sampling dan kuantisasi. Sampling membagi gambar menjadi kotak-kotak kordinat (matriks), sedangkan kuantisasi memberikan nilai angka pada setiap kotak tersebut berdasarkan tingkat kecerahannya. Pada format gambar 8-bit yang paling sering dipakai di OpenCV, nilai intensitas warna ini dibatasi dari angka 0 sampai 255. Nilai 0 berarti warna hitam gelap total, sedangkan 255 berarti warna putih paling terang. Pemahaman tentang matriks angka inilah yang menjadi dasar untuk melakukan deteksi warna dan perbaikan cahaya pada citra.

2.2 Perbandingan Ruang Warna RGB dan Konversi ke HSV

Kamera smartphone atau digital selalu menangkap gambar menggunakan format standar RGB (Red, Green, Blue). Format ini bekerja dengan menggabungkan cahaya merah, hijau, dan biru untuk menghasilkan berbagai macam warna. Walaupun sangat bagus untuk ditampilkan di layar monitor, RGB ternyata kurang cocok untuk analisis komputer seperti deteksi objek. Alasannya sederhana: warna RGB sangat gampang berubah kalau pencahayaannya berubah. Sedikit saja ada bayangan atau cahaya tambahan, nilai RGB dari sebuah benda bisa langsung berbeda jauh.

Untuk mengatasi masalah cahaya ini, pada praktikum ini gambar RGB diubah dulu ke dalam format HSV (Hue, Saturation, Value). Format HSV memisahkan informasi warna dari seberapa terang gambar tersebut, sehingga sangat cocok untuk deteksi warna tinta di atas kertas.

2.3 Pemisahan Objek (*Thresholding*) dan Logika *Bitwise*

Thresholding atau penentuan ambang batas adalah cara paling umum untuk memisahkan objek yang ingin kita ambil dari latar belakangnya (background). Dalam kasus ini, kita ingin mengambil tulisan tangan dan membuang warna kertas putihnya. Caranya adalah dengan menentukan batas bawah dan batas atas untuk warna merah, hijau, dan biru di format HSV.

Konsepnya begini: program akan mengecek satu per satu piksel di gambar. Kalau nilai warna piksel itu masuk ke dalam rentang batas yang kita tentukan, nilainya diubah jadi 255 (putih atau True). Kalau warnanya meleset dari rentang, nilainya diubah jadi 0 (hitam atau False). Proses ini menghasilkan gambar hitam-putih yang sering disebut mask atau topeng.

Setelah topeng ini jadi, kita tempelkan kembali ke gambar aslinya menggunakan operasi logika matematika yang disebut Bitwise AND. Sifat logika AND adalah mengalikan nilai. Jadi, area yang kena topeng putih (1) akan dikali dengan warna aslinya sehingga warnanya tetap muncul, sedangkan area yang kena topeng hitam (0) akan dikali nol sehingga berubah jadi warna hitam pekat.

Khusus untuk warna merah, prosesnya sedikit berbeda. Di dalam roda warna HSV, warna merah itu letaknya ada di ujung awal dan ujung akhir (melintasi sudut nol). Oleh karena itu, kita harus bikin dua batas threshold yang berbeda untuk warna merah. Kedua batas ini kemudian digabung menggunakan operasi logika Bitwise OR. Logika OR berfungsi agar asalkan piksel itu masuk ke salah satu batas merah tadi, program tetap mengenalinya sebagai bagian dari tulisan tangan.

2.4 Membaca Grafik Penyebaran Piksel (Histogram)

Histogram citra adalah grafik yang dipakai untuk melihat sebaran warna terang dan gelap di dalam sebuah gambar. Sumbu X (mendatar) pada grafik menunjukkan nilai intensitas warna dari 0 (hitam) sampai 255 (putih), sedangkan sumbu Y (tegak) menunjukkan jumlah pikselnya.

Menganalisis histogram itu sangat penting untuk membuktikan apakah deteksi warna kita berhasil atau tidak. Misalnya, gambar kertas putih polos akan punya histogram yang grafiknya menumpuk di sisi kanan (karena putih nilainya mendekati 255). Setelah kita lakukan *thresholding* dan *Bitwise AND*, latar belakang kertasnya berubah jadi hitam. Hasilnya, grafik di sebelah kanan akan hilang dan pindah ke angka 0. Puncak grafik yang tersisa di tengah-tengah adalah murni grafik dari warna tinta tulisan tangan yang berhasil kita deteksi.

2.5 Pengubahan ke Format *Grayscale*

Sebelum memperbaiki foto *selfie* yang terkena *backlight*, gambar berwarna harus diubah dulu jadi gambar abu-abu (*grayscale*). Tujuannya supaya komputer prosesnya lebih ringan dan tidak merusak susunan warna asli gambar pas kontrasnya diubah.

Mengubah ke *grayscale* itu bukan sekadar mengambil nilai rata-rata dari merah, hijau, dan biru. Mata manusia itu lebih peka terhadap warna hijau daripada warna lainnya. Oleh karena itu, hitungan rumusnya memberikan bobot paling besar di warna hijau: $Y = 0.299R + 0.587G + 0.114B$. Dengan rumus ini, hasil gambar abu-abunya akan terlihat lebih natural dan pas dengan apa yang dilihat langsung oleh mata kita.

2.6. Memperbaiki Citra *Backlight* dengan Metode CLAHE

Foto *backlight* terjadi ketika cahaya dari belakang subjek jauh lebih terang daripada cahaya yang mengenai wajah. Akibatnya, wajah di foto jadi terlihat gelap seperti siluet. Kalau kita pakai cara biasa buat menaikkan kecerahan atau kontras secara keseluruhan (*Global Histogram Equalization* / GHE), semua bagian gambar akan ikut terang. Wajahnya memang jadi kelihatan, tapi langit atau pohon di *background* malah jadi terlalu terang sampai warnanya hilang (*color burn* atau keputihan), dan fotonya jadi banyak *noise* (berbintik).

Solusi terbaik untuk masalah ini adalah algoritma CLAHE (*Contrast Limited Adaptive Histogram Equalization*). Cara kerja CLAHE jauh lebih pintar karena dia tidak memproses seluruh gambar sekaligus. Gambar akan dipotong-potong dulu ke dalam bentuk kotak-kotak kecil (*grid/tiles*), biasanya ukurannya 8 X 8 piksel. Kemudian, kontras dari setiap kotak kecil itu diperbaiki sendiri-sendiri.

Keunggulan utama CLAHE ada di bagian *Contrast Limiting* (pembatasan kontras). Jadi, kalau ada bagian grafik di sebuah kotak yang nilainya terlalu tinggi (melewati batas ambang/ *clip limit*), grafiknya akan dipotong. Sisa potongan ini tidak dibuang, melainkan disebar rata ke area lain di sekitarnya. Hal ini berguna supaya gambarnya tidak *over-exposure* dan mencegah timbulnya *noise*.

Setelah semua kotak kecil selesai diperbaiki kontrasnya, CLAHE akan menjahit kotak-kotak tersebut kembali menjadi gambar utuh. Supaya garis potongan antar kotaknya tidak kelihatan belang, program menggunakan teknik interpolasi bilinear untuk menghaluskan batas-batasnya. Hasil akhirnya, wajah yang gelap jadi lebih terang dan detail, tapi pemandangan di belakangnya tetap natural dan tidak terbakar warna.

BAB III
HASIL

3.1 Pembahasan Soal 1
Library yang digunakan:

```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt
```

Kita akan menggunakan Library itu semua untuk Menjalankan program kita kali ini. Adapun beberapa programnya sebagai berikut:

Mendeteksi BGR Pada Gambar, Script:

```
fig, axs = plt.subplots(2,2, figsize=(10, 8)) #202431129_Gian Ananda Virja

axs[0,0].imshow(kontras, cmap='gray')
axs[0,0].set_title("Gambar Asli")
axs[0,0].axis('off')

axs[0,1].imshow(b, cmap='gray')
axs[0,1].set_title("Gambar Biru")
axs[0,1].axis('off')

axs[1,0].imshow(g, cmap='gray')
axs[1,0].set_title("Gambar Hijau")
axs[1,0].axis('off')

axs[1,1].imshow(r, cmap='gray')
axs[1,1].set_title("Gambar Merah")
axs[1,1].axis('off')

plt.tight_layout()
plt.show()
```

Program mengubah format warna dari BGR menjadi RGB menggunakan fungsi cv2.cvtColor(). Selanjutnya citra dipisahkan menjadi tiga channel warna yaitu biru, hijau, dan merah menggunakan indexing array. Masing-masing channel kemudian ditampilkan menggunakan imshow() dan otomatis di grayscale menggunakan cmap='gray'.

Output:

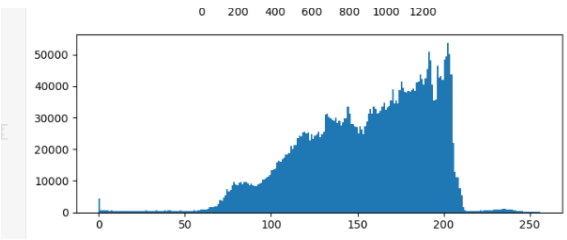
Analisis Output: Hasil ekstraksi citra menunjukkan perbedaan kontras yang signifikan antara gambar asli dengan ketiga varian warna teks. Pada **channel biru**, teks berwarna biru memiliki intensitas paling tinggi, begitu pula dengan **channel merah** dan **hijau** yang menampilkan kecerahan maksimal pada teks dengan warna yang bersesuaian. Fenomena ini membuktikan bahwa separasi saluran warna telah berjalan efektif berdasarkan nilai intensitas piksel tiap warna.

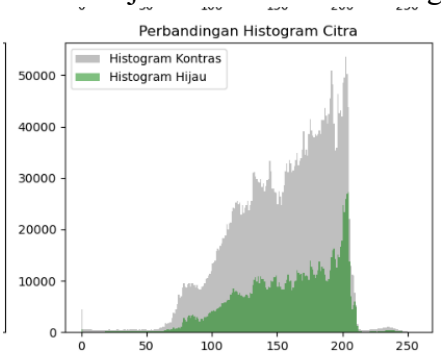
Script Perbandingan Semua Histogram Citra:

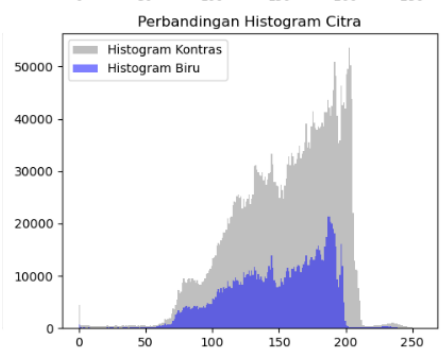
```python
import cv2 as cv
import matplotlib.pyplot as plt #202431129_Gian Ananda Virja

gambar_murni = cv.imread('tulisan_gian.jpeg')
citra_rgb = cv.cvtColor(gambar_murni, cv.COLOR_BGR2RGB)

b_channel = gambar_murni[:, :, 0]
g_channel = gambar_murni[:, :, 1]
r_channel = gambar_murni[:, :, 2]

hist_kontras = cv.calcHist([citra_rgb], [0], None, [256], [0, 256])
hist_blue = cv.calcHist([b_channel], [0], None, [256], [0, 256])
hist_red = cv.calcHist([r_channel], [0], None, [256], [0, 256])
hist_green = cv.calcHist([g_channel], [0], None, [256], [0, 256])

fig, axs = plt.subplots(2, figsize=(10, 8))

axs[0].set_title("Perbandingan Semua Histogram Citra")
axs[0].hist(citra_rgb.ravel(), bins=256, range=[0, 256], color='yellow', alpha=0.5, label="Histogram Kontras")
axs[0].hist(b_channel.ravel(), bins=256, range=[0, 256], color='blue', alpha=0.5, label="Histogram Biru")
axs[0].hist(r_channel.ravel(), bins=256, range=[0, 256], color='red', alpha=0.5, label="Histogram Merah")
axs[0].hist(g_channel.ravel(), bins=256, range=[0, 256], color='lime', alpha=0.5, label="Histogram Hijau")
axs[0].legend()

axs[1].plot(hist_kontras, color='yellow', alpha=0.5, label="Histogram Kontras")
axs[1].plot(hist_blue, color='blue', alpha=0.5, label="Histogram Biru")
axs[1].plot(hist_red, color='red', alpha=0.5, label="Histogram Merah")
axs[1].plot(hist_green, color='lime', alpha=0.5, label="Histogram Hijau")
axs[1].legend()

plt.tight_layout()
plt.show()
```

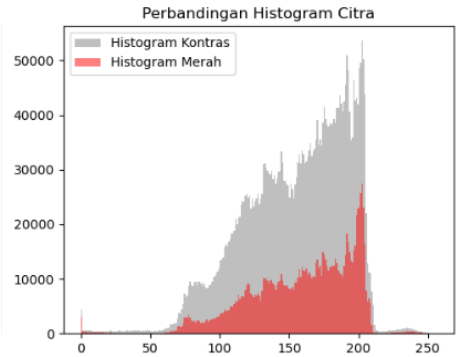
Program ini berhasil melakukan ekstraksi informasi citra dengan memisahkan saluran warna (RGB) dan menghitung distribusi intensitas pikselnya melalui fungsi cv.calcHist, di mana hasil visualisasi menunjukkan bahwa setiap objek warna mencapai tingkat kecerahan tertinggi pada kanal yang bersesuaian (misalnya, teks biru tampak paling terang pada *channel* biru). Fenomena ini secara teknis memvalidasi bahwa proses pemisahan saluran warna telah berjalan akurat sesuai dengan distribusi nilai intensitas piksel asli pada citra tersebut.

OUTPUT HISTOGRAM:









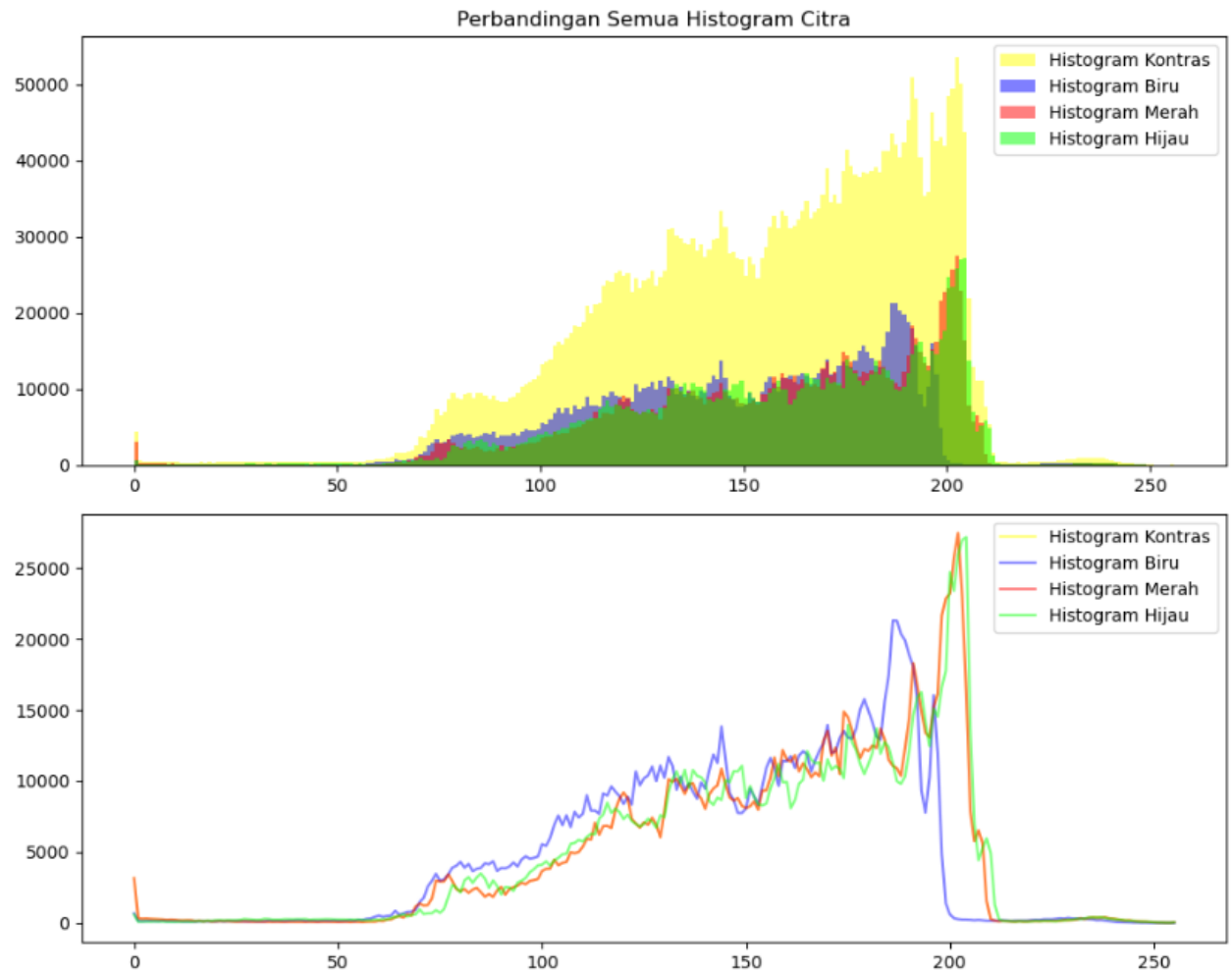
Analisis Histogram:

Keempat histogram menunjukkan konsentrasi piksel yang dominan pada rentang nilai intensitas 100 hingga 200. Hal ini mengindikasikan bahwa citra tersebut secara keseluruhan memiliki tingkat pencahayaan yang cukup terang (area midtone ke highlight).

Distribusi pada kanal hijau menunjukkan puncak intensitas yang cukup tajam di sekitar nilai **200**. Frekuensi piksel hijau yang menonjol di area terang ini menandakan adanya elemen warna hijau yang sangat cerah pada citra asli.

Berbeda dengan kanal lainnya, kanal biru memiliki lonjakan frekuensi yang signifikan di sekitar intensitas **185** sebelum mengalami penurunan tajam. Ini menunjukkan bahwa komponen warna biru tersebar luas namun tidak mencapai tingkat kecerahan maksimal sekstrem kanal merah atau hijau.

Kanal merah menampilkan pola distribusi yang sangat mirip dengan kanal hijau, dengan puncak frekuensi tertinggi berada di atas nilai **200**. Kesamaan pola antara merah dan hijau menunjukkan bahwa kedua warna ini kemungkinan besar bergabung membentuk area terang yang seragam dalam citra.



Analisis Histogram:

Grafik menunjukkan bahwa mayoritas piksel dari ketiga saluran warna (biru, merah, dan hijau) terkonsentrasi pada rentang intensitas **100 hingga 200**. Hal ini mengindikasikan bahwa citra tersebut memiliki tingkat pencahayaan yang cukup dominan atau cenderung terang.

Saluran **biru** terlihat memiliki puncak frekuensi yang lebih awal (sekitar intensitas 185) dibandingkan saluran **merah** dan **hijau** yang mencapai puncaknya secara bersamaan di area yang lebih terang, yaitu di atas intensitas 200. Perbedaan posisi puncak ini mencerminkan variasi kontras dan komposisi warna yang unik pada masing-masing saluran.

Histogram kontras secara keseluruhan menunjukkan akumulasi dari ketiga saluran warna, di mana frekuensinya meningkat secara bertahap dan mencapai titik tertinggi pada nilai intensitas tinggi sebelum akhirnya menurun tajam mendekati nilai 210.

Grafik menunjukkan bahwa citra ini memiliki rentang dinamis yang terbatas pada area midtone hingga highlight. Tidak ada aktivitas piksel yang signifikan pada rentang 0 hingga 60, yang berarti citra tersebut minim area bayangan (shadow) yang benar-benar gelap atau hitam pekat.

Terdapat penurunan drastis pada frekuensi piksel tepat setelah intensitas 210. Hal ini mengindikasikan bahwa citra tidak mengalami *clipping* pada area putih (tidak ada detail yang hilang karena terlalu terang), karena distribusi tidak menumpuk di ujung kanan (nilai 255).

LANJUT MENCARI NILAI AMBANG:

```
import cv2 as cv #202431129_Gian Ananda Virja
import numpy as np
import matplotlib.pyplot as plt

gambar_tulisan = cv.imread('tulisan_gian.jpeg')
b = gambar_tulisan[:, :, 0]
g = gambar_tulisan[:, :, 1]
r = gambar_tulisan[:, :, 2]

panjang = len(gambar_tulisan)
lebar = len(gambar_tulisan[0])
```

Kita menggunakan Import terlebih dahulu agar kita bisa menggunakan library yang kita gunakan seperti misalkan matplotlib digunakan untuk perhitungan matematis nantinya untuk menghitung warna.

NILAI AMBANG NONE

```
ambang = 255 #202431129_Gian Ananda Virja
gambar_none = np.zeros((panjang, lebar), dtype=np.uint8)

for i in range(panjang):
    for j in range(lebar):
        nilai_gray = int(0.299 * r[i,j] + 0.587 * g[i,j] + 0.114 * b[i,j])
        if nilai_gray > ambang:
            gambar_none[i, j] = 1
        else:
            gambar_none[i, j] = 0
```

Yang pertama kita buat NILAI AMBANG untuk NONE terlebih dahulu, gunanya kita akan membandingkan ke empat gambar yang akan kita bandingkan.

NILAI AMBANG BIRU DAN MERAH

```
ambang_bawah_merah = 120 #202431129_Gian Ananda Virja
ambang_atas_merah = 255
gambar_rb = np.zeros((panjang, lebar), dtype=np.uint8)

for i in range(panjang):
    for j in range(lebar):
        biru_flag = ambang_bawah_biru <= b[i, j] <= ambang_atas_biru and r[i, j] < 100 and g[i, j] < 100
        merah_flag = ambang_bawah_merah <= r[i, j] <= ambang_atas_merah and g[i, j] < 100 and b[i, j] < 100
        if merah_flag or biru_flag:
            gambar_rb[i, j] = 1
        else:
            gambar_rb[i, j] = 0
```

Kita lanjut mencari nilai ambang biru dan merah, Logika pemrograman ini berfungsi sebagai **penyaring warna otomatis** yang bertugas menyeleksi hanya warna biru dan merah murni dari sebuah gambar. Program akan memindai setiap piksel dalam gambar dan menerapkan aturan ketat melalui variabel ambang_batas: sebuah piksel dianggap **biru_flag** atau **merah_flag** jika komponen warna utamanya berada dalam rentang terang (misalnya di atas 120), sementara dua komponen warna lainnya harus sangat rendah (di bawah 100) untuk memastikan warna tersebut tidak tercampur. Jika sebuah piksel berhasil memenuhi kriteria warna merah atau biru tersebut, program akan menandainya dengan nilai **1** pada variabel gambar_rb, sedangkan piksel yang tidak memenuhi syarat akan dibuang atau ditandai dengan nilai **0**. Hasil akhir dari proses ini adalah pemisahan objek berwarna spesifik dari latar belakangnya dalam bentuk data biner.

NIlAI AMBANG BIRU, HIJAU DAN MERAH SERTA HASIL CITRA

```
ambang_bawah_hijau = 95 #202431129_Gian Ananda Virja
ambang_atas_hijau = 255
gambar_rgb = np.zeros((panjang, lebar), dtype=np.uint8)

for i in range(panjang):
    for j in range(lebar):
        biru_flag = ambang_bawah_biru <= b[i, j] <= ambang_atas_biru and r[i, j] < 100 and g[i, j] < 100
        merah_flag = ambang_bawah_merah <= r[i, j] <= ambang_atas_merah and g[i, j] < 100 and b[i, j] < 100
        hijau_flag = ambang_bawah_hijau <= g[i, j] <= ambang_atas_hijau and b[i, j] < 100 and r[i, j] < 100
        if biru_flag or merah_flag or hijau_flag:
            gambar_rgb[i, j] = 1
        else:
            gambar_rgb[i, j] = 0

fig, axs = plt.subplots(2, 2, figsize=(12, 8))

axs[0, 0].imshow(gambar_none, cmap='gray')
axs[0, 0].set_title("NONE")
axs[0, 0].axis('off')

axs[0, 1].imshow(gambar_b, cmap='gray')
axs[0, 1].set_title("BLUE")
axs[0, 1].axis('off')

axs[1, 0].imshow(gambar_rb, cmap='gray')
axs[1, 0].set_title("RED-BLUE")
axs[1, 0].axis('off')

axs[1, 1].imshow(gambar_rgb, cmap='gray')
axs[1, 1].set_title("RED-GREEN-BLUE")
axs[1, 1].axis('off')

plt.tight_layout()
plt.show()
```

Progam ini berfungsi untuk mendeteksi dan menggabungkan tiga warna utama Merah hijau dan biru, menjadi satu hasil citra biner. Program ini bekerja dengan cara memindai setiap piksel dan menerapkan kriteria seleksi yang ketat: sebuah piksel akan ditandai sebagai warna murni (biru_flag, merah_flag, atau hijau_flag) jika intensitas warna tersebut berada dalam rentang ambang batas yang ditentukan (misalnya hijau antara 95 hingga 255), sementara dua komponen warna pendamping lainnya harus bernilai rendah atau di bawah 100. Jika piksel tersebut memenuhi salah satu dari ketiga kriteria warna tersebut, maka akan diberi nilai **1**, dan jika tidak akan diberi nilai **0**, yang kemudian hasilnya divisualisasikan dalam empat perbandingan panel (NONE, BLUE, RED-BLUE, dan RED-GREEN-BLUE) untuk menunjukkan bagaimana setiap warna ditambahkan secara bertahap ke dalam hasil akhir citra.

Outputnya:

NONE: Panel ini menampilkan citra yang sepenuhnya hitam, mengindikasikan kondisi awal di mana belum ada kriteria warna yang diterapkan atau lolos seleksi.

BLUE: Hasil ini menunjukkan keberhasilan filter dalam mendeteksi objek berwarna biru, yang memunculkan teks **"ANANDA"** di tengah layar sebagai area berwarna putih (nilai piksel 1).

RED-BLUE: Pada tahap ini, filter warna merah ditambahkan ke dalam sistem, sehingga teks **"GIAN"** muncul mendampingi teks sebelumnya, membentuk rangkaian kata **"GIAN ANANDA"**.

RED-GREEN-BLUE: Ini merupakan hasil akhir penggabungan ketiga filter warna utama. Penambahan filter hijau berhasil memunculkan teks **"VIRJA"** serta beberapa detail noise/objek tambahan di bagian tepi gambar, melengkapi seluruh teks menjadi **"GIAN ANANDA VIRJA"**.

3.2 Pembahasan Soal 2 (Memperbaiki Backlight)

```
import cv2 as cv #202431129_Gian Ananda Virja
import numpy as np
import matplotlib.pyplot as plt

gambar_asli = cv.imread('selfie_gian.jpeg')
r, g, b = gambar_asli[:,:,2], gambar_asli[:,:,1], gambar_asli[:,:,0]
panjang = len(gambar_asli)
lebar = len(gambar_asli[0])
```

Kodingan ini berfungsi sebagai tahap inisialisasi yang melakukan pemanggilan pustaka utama (OpenCV, NumPy, dan Matplotlib), pembacaan file citra 'selfie_gian.jpeg', serta pemisahan komponen warna dasar menjadi variabel r, g, dan b sekaligus mengidentifikasi dimensi panjang dan lebar piksel citra agar siap untuk diproses pada tahap penyaringan warna selanjutnya.

```
gambar_gray = np.zeros((panjang, lebar), dtype=np.uint8) #202431129_Gian Ananda Virja
for i in range(panjang):
    for j in range(lebar):
        gambar_gray[i,j] = int(0.299 * r[i,j] + 0.587 * g[i,j] + 0.114 * b[i,j])
```

Kodingan ini berfungsi untuk mengubah gambar berwarna menjadi hitam-putih (abu-abu) dengan cara membuat kanvas kosong berukuran sama menggunakan np.zeros dan mengisinya melalui perhitungan matematis pada setiap piksel. Program menggunakan rumus bobot standar pencahayaan (luminositas) yaitu 0.299 * R + 0.587 * G + 0.114 * B untuk menggabungkan nilai merah, hijau, dan biru menjadi satu nilai intensitas abu-abu yang seimbang. Prosedur ini memastikan bahwa transisi dari warna ke abu-abu tetap terlihat alami bagi mata manusia karena mempertimbangkan perbedaan sensitivitas mata terhadap tiap warna dasar.

```
gambar_cerah = np.zeros((panjang, lebar), dtype=np.uint8) #202431129_Gian Ananda Virja
gamma = 1.4
for i in range(panjang):
    for j in range(lebar):
        hasil = ((gambar_gray[i,j] / 255.0) ** (1.0 / gamma)) * 255
        gambar_cerah[i,j] = np.clip(hasil, 0, 255)
```

Kodingan ini berfungsi untuk mencerahkan gambar secara non-linear dengan menerapkan metode Koreksi Gamma pada setiap piksel gambar grayscale. Program menggunakan nilai gamma = 1.4 dan menghitung intensitas baru. Proses ini diawali dengan normalisasi nilai piksel ke rentang 0-1, melakukan transformasi pangkat untuk menaikkan intensitas pada area yang gelap, dan diakhiri dengan fungsi np.clip untuk memastikan nilai akhir piksel tetap berada dalam batas standar 0 hingga 255.

```
gambar_kontras = np.zeros((panjang, lebar), dtype=np.uint8) #202431129_Gian Ananda Virja
kontras = 1.25
for i in range(panjang):
    for j in range(lebar):
        hasil = 128 + (int(gambar_gray[i,j]) - 128) * kontras
        gambar_kontras[i,j] = np.clip(hasil, 0, 255)
```

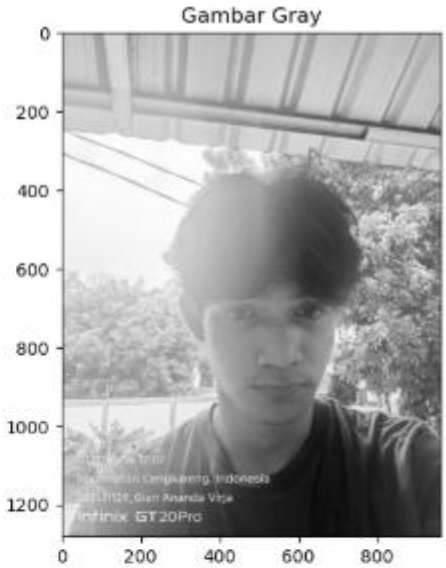
Kodingan ini berfungsi untuk mempertajam perbedaan antara area gelap dan terang pada gambar dengan menerapkan pengali kontras sebesar **1.25** terhadap titik tengah intensitas warna (nilai 128). program akan menarik nilai piksel yang lebih besar dari 128 menjadi lebih terang dan nilai yang lebih kecil dari 128 menjadi lebih gelap. Di akhir proses, fungsi np.clip digunakan untuk memastikan nilai piksel tetap berada dalam rentang standar **0 hingga 255** agar gambar tidak mengalami kerusakan data (*overflow*).

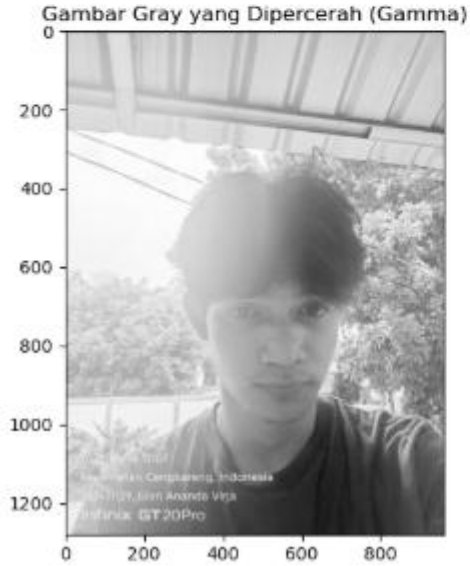
```
gambar_final = np.zeros((panjang, lebar), dtype=np.uint8) #202431129_Gian Ananda Virja
gamma_f = 1.25
kontras_f = 1.15
for i in range(panjang):
    for j in range(lebar):
        cerah_f = ((gambar_gray[i,j] / 255.0) ** (1.0 / gamma_f)) * 255
        hasil = 128 + (cerah_f - 128) * kontras_f
        gambar_final[i,j] = np.clip(hasil, 0, 255)

fig, axs = plt.subplots(3, 2, figsize=(11, 14))
axs[0,0].imshow(cv.cvtColor(gambar_asli, cv.COLOR_BGR2RGB))
axs[0,0].set_title("Gambar Asli")
axs[0,1].imshow(gambar_gray, cmap='gray')
axs[0,1].set_title("Gambar Gray")
axs[1,0].imshow(gambar_cerah, cmap='gray')
axs[1,0].set_title("Gambar Gray yang Dipercerah (Gamma)")
axs[1,1].imshow(gambar_kontras, cmap='gray')
axs[1,1].set_title("Gambar Gray yang Diperkontras (Midpoint)")
axs[2,0].imshow(gambar_final, cmap='gray')
axs[2,0].set_title("Gambar Gray yang Dipercerah dan Diperkontras")
axs[2,1].axis('off')
plt.tight_layout()
plt.show()
```

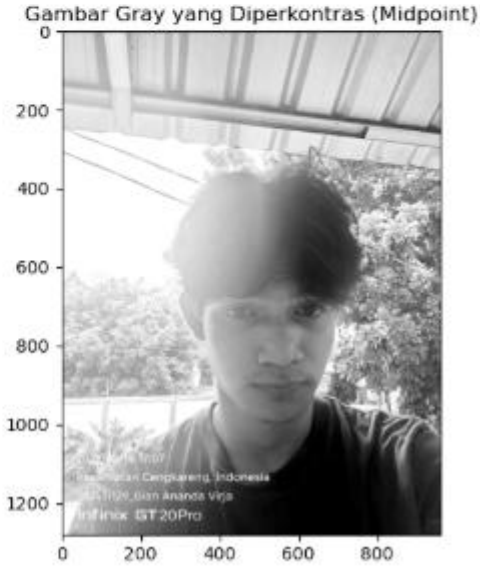
Kodingan ini berfungsi untuk menghasilkan citra akhir (gambar_final) dengan menggabungkan dua teknik perbaikan kualitas sekaligus, yaitu mencerahkan gambar menggunakan **Koreksi Gamma** (gamma_f = 1.25) dan mempertajam detail melalui **penyesuaian kontras** (kontras_f = 1.15). Proses ini dilakukan dengan menghitung ulang setiap piksel dari citra grayscale, memastikan hasilnya tetap dalam rentang 0-255 dengan fungsi np.clip, dan diakhiri dengan menyajikan visualisasi komparatif dalam enam panel gambar (Asli, Gray, Hasil Cerah, Hasil Kontras, dan Hasil Akhir) untuk mempermudah analisis perbedaan efektivitas dari masing-masing metode pengolahan yang telah diterapkan.

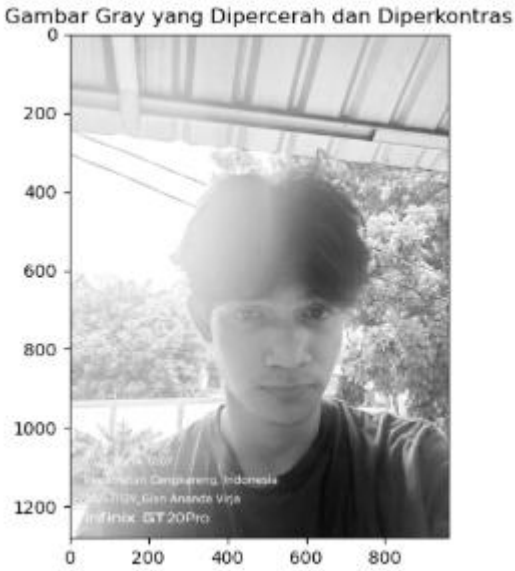
Outputnya:

Gambar Asli
0
200
400
600
800
1000
1200
0
200
400
600
800


Gambar Gray
0
200
400
600
800
1000
1200
0
200
400
600
800


Gambar Gray yang Dipercerah (Gamma)
0
200
400
600
800
1000
1200
0
200
400
600
800


Gambar Gray yang Diperkontras (Midpoint)
0
200
400
600
800
1000
1200
0
200
400
600
800


Gambar Gray yang Dipercerah dan Diperkontras
0
200
400
600
800
1000
1200
0
200
400
600
800

Berdasarkan perbandingan visual pada Gambar, terlihat transisi bertahap dari citra asli ke bentuk grayscale, di mana penerapan **Koreksi Gamma** berhasil meningkatkan visibilitas detail pada area gelap (bayangan), sementara teknik **Midpoint Contrast** mempertajam perbedaan antara elemen terang dan gelap; hasil akhirnya menunjukkan bahwa kombinasi kedua metode tersebut pada panel "Gambar Gray yang Dipercerah dan Diperkontras" menghasilkan kualitas visual yang paling optimal dengan tingkat kecerahan yang seimbang dan detail subjek yang lebih tegas.

Berdasarkan hasil praktikum soal kedua, proses perbaikan citra backlight menggunakan operasi pixel berhasil dilakukan dengan mengubah citra menjadi grayscale serta meningkatkan brightness dan contrast pada gambar. Konversi grayscale membantu menyederhanakan proses pengolahan citra karena gambar hanya memiliki satu channel intensitas. Peningkatan brightness membuat citra menjadi lebih terang sehingga objek pada gambar lebih mudah terlihat, sedangkan peningkatan contrast membantu memperjelas perbedaan antara area gelap dan terang pada citra. Hasil akhir menunjukkan bahwa kombinasi brightness dan contrast mampu memperbaiki kualitas visual citra backlight meskipun efek cahaya dari latar belakang masih tetap terlihat. Praktikum ini membuktikan bahwa operasi pixel sederhana menggunakan OpenCV dapat digunakan untuk meningkatkan kualitas citra digital dan membantu memperjelas objek pada kondisi pencahayaan yang kurang ideal.

BAB IV
PENUTUP

Berdasarkan landasan teori dan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa pengolahan citra digital merupakan teknik yang digunakan untuk mengolah dan memperbaiki kualitas citra menggunakan bantuan komputer. Dalam praktikum ini, OpenCV dan Python berhasil digunakan untuk melakukan berbagai proses pengolahan citra seperti pemisahan channel warna, analisis histogram, thresholding, grayscale, brightness, dan contrast. Pada soal pertama, metode thresholding berhasil digunakan untuk mendeteksi warna biru, merah, dan hijau berdasarkan nilai ambang tertentu pada setiap channel warna. Histogram juga membantu dalam menganalisis distribusi intensitas pixel pada citra sehingga perbedaan karakteristik warna dapat terlihat dengan lebih jelas. Pada soal kedua, operasi pixel berupa peningkatan brightness dan contrast berhasil membantu memperbaiki kualitas citra backlight sehingga objek pada gambar menjadi lebih terlihat dibandingkan citra awal. Secara keseluruhan, praktikum ini menunjukkan bahwa teknik pengolahan citra digital dapat digunakan untuk membantu proses analisis visual, deteksi warna, dan perbaikan kualitas gambar pada berbagai kondisi pencahayaan.

DAFTAR PUSTAKA

Pratama, A. R., & Syafiq, M. (2024). "Optimasi Segmentasi Objek Berbasis Ruang Warna HSV Menggunakan Operasi Morfologi pada Pemrosesan Citra Digital". Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI), 13(1), 45-52.

Hidayatullah, R., & Wibowo, A. (2025). "Implementasi Metode Thresholding dan Logika Boolean untuk Ekstraksi Fitur Teks pada Dokumen Pindaian Terdegradasi". Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK), 12(2), 112-119.

Setiawan, D., Kusumawati, R., & Wijaya, B. (2023). "Analisis Perbandingan Transformasi Grayscale dan Ruang Warna YCbCr pada Pra-Pemrosesan Pengenalan Pola Wajah". Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi), 7(4), 670-678.

Yulianti, S., & Darmawan, I. (2024). "Peningkatan Kualitas Citra Medis Menggunakan Contrast Limited Adaptive Histogram Equalization (CLAHE) untuk Meminimalisir Amplifikasi Noise". Jurnal Rekayasa Elektrika, 20(3), 180-188.

Ardiansyah, F., dkk. (2025). "Evaluasi Ekualisasi Histogram Global dan CLAHE dalam Penanganan Distorsi Pencahayaan Backlight pada Sistem Pengawasan Pintar". Jurnal Ilmiah Teknologi Sistem Informasi, 11(1), 22-30.

Raharjo, B. P., & Purnomo, H. (2026). "Pemodelan Matematis Filter Spasial dan CLAHE pada Perbaikan Citra Digital Under-Exposed". Jurnal Ilmiah Komputasi dan Informatika (JIKI), 9(2), 105-115.